

گزارش تمرین چهارم

تولید تصویر چهره با شبکه مولد تخصصی عمیق روی CelebA

کاری از : سارا محمدی

شرح مسئله

هدف، طراحی و آموزش یک DCGAN با Keras برای تولید چهره‌های جدید از بردارهای نویز تصادفی است. مجموعه داده CelebA دریافت می‌شود، تصاویر به اندازه 64×64 آماده می‌شوند و Generator و Discriminator طی ۳۰ دوره به صورت رقابتی آموزش می‌بینند.

روش حل

Generator بردار ۱۲۸ بعدی را با Dense و Conv2DTranspose به تصویر RGB تبدیل می‌کند. Discriminator با لایه‌های Conv2D واقعی یا ساختگی بودن تصویر را تشخیص می‌دهد. هموارسازی برچسب واقعی، Adam با $\beta_1=0.5$ و ثابت چهره‌های ثابت در پایان هر دوره برای پایدارسازی و ارزیابی روند آموزش استفاده شده‌اند.

۱. نصب وابستگی‌ها

```
!pip -q install kagglehub imageio
```

۲. کتابخانه‌ها و بررسی محیط اجرا

```
import os
import math
import random
from pathlib import Path

import imageio.v2 as imageio
import kagglehub
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from IPython.display import Image, display
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
print('TensorFlow:', tf.__version__)
print('GPU:', tf.config.list_physical_devices('GPU'))
```

```
TensorFlow: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

۳. تنظیم دسترسی Kaggle

```
from google.colab import files
import shutil

uploaded = files.upload()
os.makedirs('/root/.kaggle', exist_ok=True)
shutil.move('kaggle.json', '/root/.kaggle/kaggle.json')
os.chmod('/root/.kaggle/kaggle.json', 0o600)
```

Saving kaggle.json to kaggle.json

به دلیل حجم بالا دیتاست به کمک kaggle در نوت بوک آن را دانلود و استفاده میکنیم با فایل جیسون احراز هویت kaggle

۴. دریافت CelebA و شناسایی تصاویر

```
DATASET_HANDLE = 'zuozhaorui/celeba'
dataset_root = Path(kagglehub.dataset_download(DATASET_HANDLE))
print('Dataset downloaded to:', dataset_root)

image_paths = sorted(dataset_root.rglob('*.jpg'))
print('Total images found:', len(image_paths))
assert len(image_paths) > 0, 'No images were found in the downloaded dataset.'
print('First image:', image_paths[0])
```

Downloading to /root/.cache/kagglehub/datasets/zuozhaorui/celeba/1.archive...

```
100%|â-â-â-â-â-â-â-â-â-â-| 2.64G/2.64G [02:08<00:00, 22.1MB/s]
Extracting files...
```

```
Dataset downloaded to: /root/.cache/kagglehub/datasets/zuozhaorui/celeba/versions/1
Total images found: 405198
First image: /root/.cache/kagglehub/datasets/zuozhaorui/celeba/versions/1/img_align_celeba/000001.jpg
```

مجموعه CelebA دریافت شده است. در مجموع ۴۰۵۱۹۸ فایل JPG شناسایی می‌شود و مسیر نخستین تصویر نیز نمایش داده شده است.

۵. آماده‌سازی خط لوله داده

```
IMAGE_SIZE = 64
CHANNELS = 3
BATCH_SIZE = 128
LATENT_DIM = 128
MAX_IMAGES = 50_000
EPOCHS = 30

random.shuffle(image_paths)
image_paths = image_paths[:MAX_IMAGES]
print('Number of photos used:', len(image_paths))

def load_and_preprocess(path):
    image = tf.io.read_file(path)
    image = tf.io.decode_jpeg(image, channels=CHANNELS)
    image = tf.image.resize(image, [IMAGE_SIZE, IMAGE_SIZE])
    image = (tf.cast(image, tf.float32) - 127.5) / 127.5
    return image

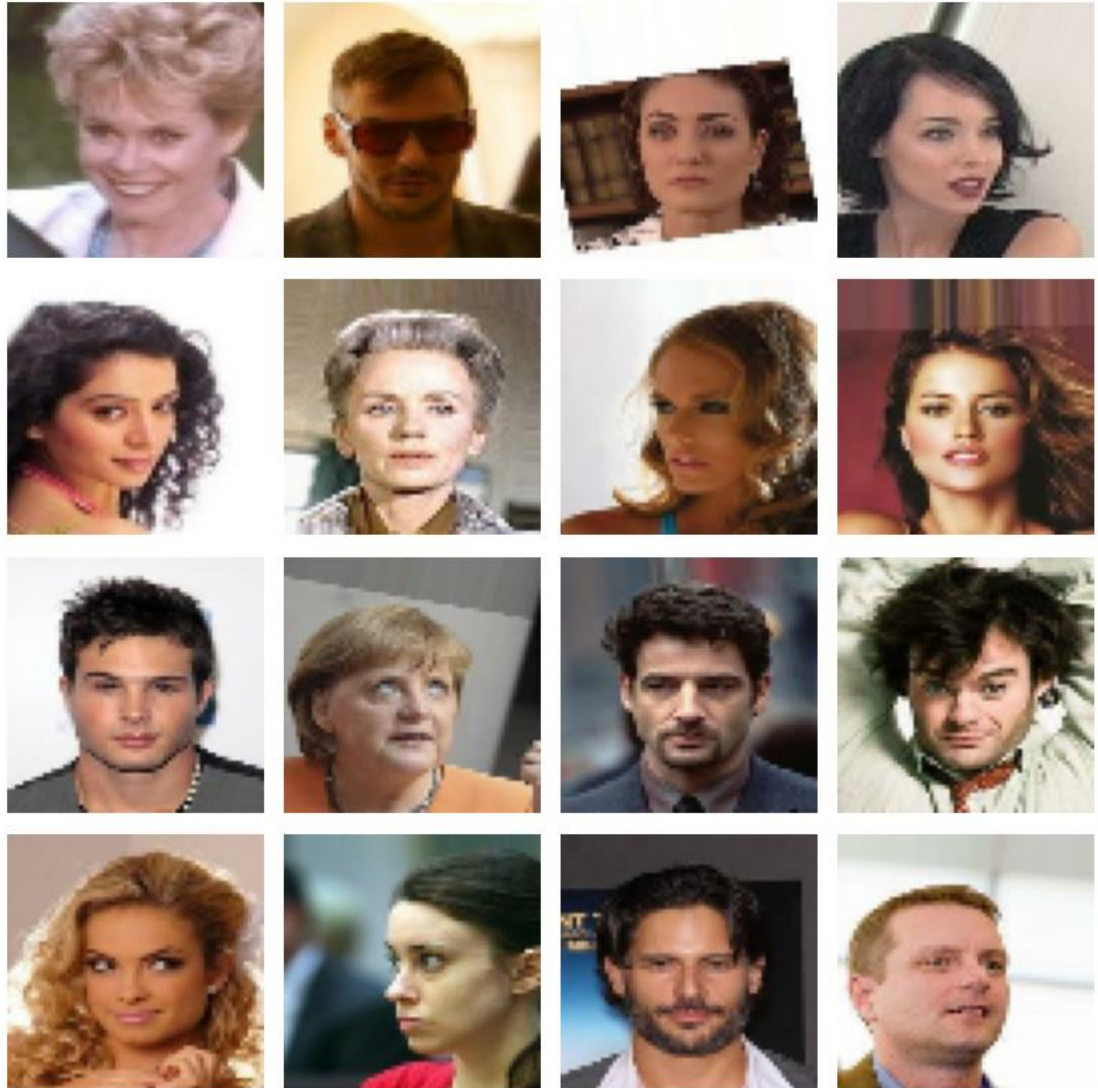
train_ds = tf.data.Dataset.from_tensor_slices([str(p) for p in image_paths])
train_ds = train_ds.shuffle(10_000, seed=0)
train_ds = train_ds.map(load_and_preprocess, num_parallel_calls=tf.data.AUTOTUNE)
train_ds = train_ds.batch(BATCH_SIZE, drop_remainder=True).prefetch(tf.data.AUTOTUNE)
```

Number of photos used: 50000

از ۵۰۰۰۰ تصویر استفاده می‌شود. تصاویر به 64×64 تغییر اندازه یافته و به بازه -1 تا 1 نرمال می‌شوند. اندازه دسته ۱۲۸ است.

۶. نمایش تصاویر واقعی آموزش

```
real_batch = next(iter(train_ds))
fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for ax, image in zip(axes.ravel(), real_batch[:16]):
    ax.imshow((image.numpy() + 1) / 2)
    ax.axis('off')
plt.tight_layout()
plt.show()
```



شانزده تصویر واقعی از داده‌های نرمال‌شده برای کنترل ورودی مدل نمایش داده شده‌اند.

۷. ساخت Generator

```
def build_generator():
    model = keras.Sequential(name='generator')
    model.add(keras.Input(shape=(LATENT_DIM,)))
    model.add(layers.Dense(4 * 4 * 512, use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.ReLU())
    model.add(layers.Reshape((4, 4, 512)))

    for filters in [256, 128, 64]:
        model.add(layers.Conv2DTranspose(filters, 4, strides=2, padding='same', use_bias=False))
        model.add(layers.BatchNormalization())
        model.add(layers.ReLU())

    model.add(layers.Conv2DTranspose(CHANNELS, 4, strides=2, padding='same', activation='tanh'))
    return model

generator = build_generator()
generator.summary()
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 8192)	1,048,576
batch_normalization	(None, 8192)	32,768
reshape (Reshape)	(None, 4, 4, 512)	0
conv2d_transpose	(None, 8, 8, 256)	2,097,152
conv2d_transpose_1	(None, 16, 16, 128)	524,288
conv2d_transpose_2	(None, 32, 32, 64)	131,072
conv2d_transpose_3	(None, 64, 64, 3)	3,075

Total params: 3,838,723 (14.64 MB)
Trainable params: 3,821,443 (14.58 MB)
Non-trainable params: 17,280 (67.50 KB)

Generator با Dense، BatchNormalization، ReLU و چهار لایه Conv2DTranspose نويز را به تصوير $3 \times 64 \times 64$ تبدیل می‌کند و ۳,۸۳۸,۷۲۳ پارامتر دارد.

۸. ساخت Discriminator

```
def build_discriminator():
    model = keras.Sequential(name='discriminator')
    model.add(keras.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, CHANNELS)))
    for filters in [64, 128, 256, 512]:
        model.add(layers.Conv2D(filters, 4, strides=2, padding='same', use_bias=False))
        model.add(layers.LeakyReLU(0.2))
        model.add(layers.Dropout(0.2))
    model.add(layers.Flatten())
    model.add(layers.Dense(1))
    return model

discriminator = build_discriminator()
discriminator.summary()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 64)	3,072
conv2d_1 (Conv2D)	(None, 16, 16, 128)	131,072
conv2d_2 (Conv2D)	(None, 8, 8, 256)	524,288
conv2d_3 (Conv2D)	(None, 4, 4, 512)	2,097,152
flatten (Flatten)	(None, 8192)	0
dense_1 (Dense)	(None, 1)	8,193

Total params: 2,763,777 (10.54 MB)
Trainable params: 2,763,777 (10.54 MB)
Non-trainable params: 0 (0.00 B)

Discriminator با چهار لایه Conv2D، LeakyReLU، Dropout و یک خروجی خطی ساخته شده و ۲,۷۶۳,۷۷۷ پارامتر دارد.

۹. تعریف بهینه‌سازها و گام آموزش

```
generator_optimizer = keras.optimizers.Adam(2e-4, beta_1=0.5)
discriminator_optimizer = keras.optimizers.Adam(1e-4, beta_1=0.5)
loss_fn = keras.losses.BinaryCrossentropy(from_logits=True)

@tf.function
def train_step(real_images):
    batch_size = tf.shape(real_images)[0]
    noise = tf.random.normal((batch_size, LATENT_DIM))

    with tf.GradientTape() as disc_tape, tf.GradientTape() as gen_tape:
        fake_images = generator(noise, training=True)

        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)

        d_loss = loss_fn(tf.ones_like(real_output) * 0.9, real_output) + \
            loss_fn(tf.zeros_like(fake_output), fake_output)
        g_loss = loss_fn(tf.ones_like(fake_output), fake_output)

    d_grads = disc_tape.gradient(d_loss, discriminator.trainable_variables)
    g_grads = gen_tape.gradient(g_loss, generator.trainable_variables)
    discriminator_optimizer.apply_gradients(zip(d_grads, discriminator.trainable_variables))
    generator_optimizer.apply_gradients(zip(g_grads, generator.trainable_variables))

    return d_loss, g_loss
```

تابع `train_step` با دو `GradientTape` خطای مولد و متمایزکننده را محاسبه می‌کند. برای برچسب واقعی مقدار ۰٫۹ به کار رفته است.

۱۰. تابع ذخیره شبکه تصاویر

```
FRAME_DIR = Path('frames')
FRAME_DIR.mkdir(exist_ok=True)
fixed_noise = tf.random.normal((16, LATENT_DIM), seed=0)

def save_image_grid(images, path):
    images = np.clip((images + 1) / 2, 0, 1)
    fig, axes = plt.subplots(4, 4, figsize=(6, 6))
    for ax, image in zip(axes.ravel(), images):
        ax.imshow(image)
        ax.axis('off')
    plt.tight_layout()
    plt.savefig(path)
    plt.close(fig)
```

یک بردار نویز ثابت و تابع ذخیره آرایش ۴×۴ تعریف می‌شود تا پیشرفت مدل در دوره‌های مختلف قابل مقایسه باشد.

۱۱. آموزش DCGAN در ۳۰ دوره

```
d_losses, g_losses = [], []

for epoch in range(EPOCHS):
    epoch_d_loss, epoch_g_loss = [], []
    for real_batch in train_ds:
        d_loss, g_loss = train_step(real_batch)
        epoch_d_loss.append(d_loss)
        epoch_g_loss.append(g_loss)

    d_losses.append(np.mean(epoch_d_loss))
    g_losses.append(np.mean(epoch_g_loss))
    print(f'epoch {epoch + 1}/{EPOCHS} - d_loss: {d_losses[-1]:.3f} - g_loss: {g_losses[-1]:.3f}')

    generated = generator(fixed_noise, training=False).numpy()
    save_image_grid(generated, FRAME_DIR / f'epoch_{epoch + 1:03d}.png')
```

```
epoch 1/30 - d_loss: 1.178 - g_loss: 0.967
epoch 2/30 - d_loss: 1.155 - g_loss: 0.981
epoch 3/30 - d_loss: 1.133 - g_loss: 1.009
epoch 4/30 - d_loss: 1.098 - g_loss: 1.046
epoch 5/30 - d_loss: 1.078 - g_loss: 1.088
epoch 6/30 - d_loss: 1.058 - g_loss: 1.134
epoch 7/30 - d_loss: 1.056 - g_loss: 1.129
epoch 8/30 - d_loss: 1.054 - g_loss: 1.142
epoch 9/30 - d_loss: 1.036 - g_loss: 1.199
epoch 10/30 - d_loss: 1.063 - g_loss: 1.174
epoch 11/30 - d_loss: 1.072 - g_loss: 1.165
epoch 12/30 - d_loss: 1.084 - g_loss: 1.162
```

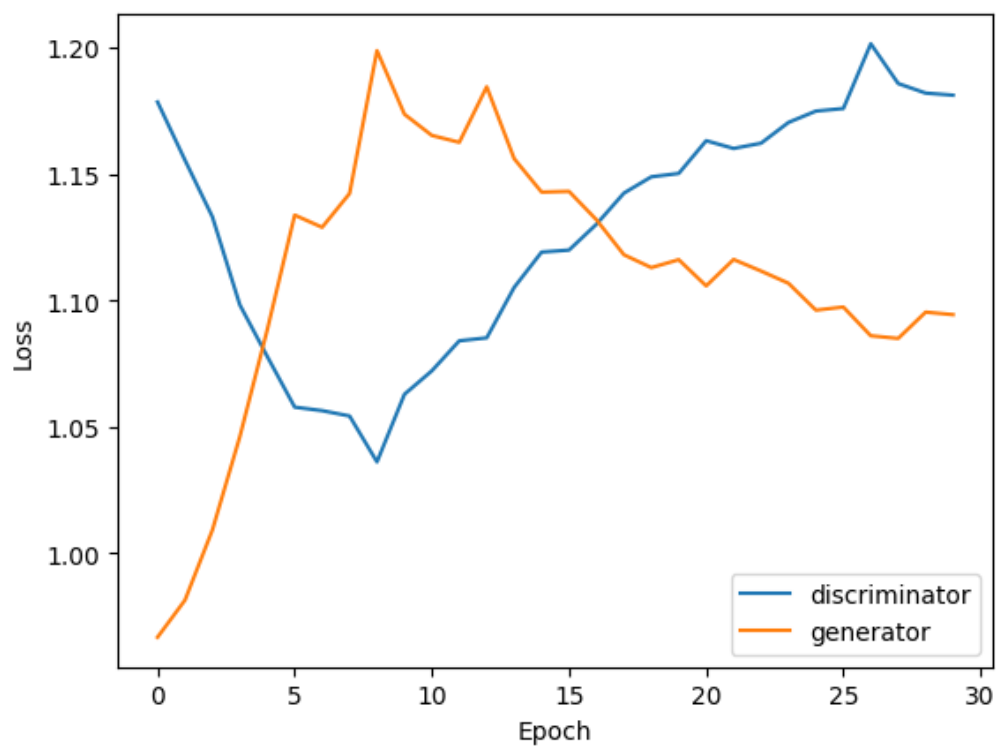
```
epoch 13/30 - d_loss: 1.085 - g_loss: 1.184
epoch 14/30 - d_loss: 1.105 - g_loss: 1.156
epoch 15/30 - d_loss: 1.119 - g_loss: 1.143
epoch 16/30 - d_loss: 1.120 - g_loss: 1.143
epoch 17/30 - d_loss: 1.130 - g_loss: 1.132
epoch 18/30 - d_loss: 1.142 - g_loss: 1.118
epoch 19/30 - d_loss: 1.149 - g_loss: 1.113
epoch 20/30 - d_loss: 1.150 - g_loss: 1.116
epoch 21/30 - d_loss: 1.163 - g_loss: 1.106
epoch 22/30 - d_loss: 1.160 - g_loss: 1.116
epoch 23/30 - d_loss: 1.162 - g_loss: 1.112
epoch 24/30 - d_loss: 1.170 - g_loss: 1.107
```

```
epoch 25/30 - d_loss: 1.175 - g_loss: 1.096
epoch 26/30 - d_loss: 1.176 - g_loss: 1.097
epoch 27/30 - d_loss: 1.201 - g_loss: 1.086
epoch 28/30 - d_loss: 1.186 - g_loss: 1.085
epoch 29/30 - d_loss: 1.182 - g_loss: 1.095
epoch 30/30 - d_loss: 1.181 - g_loss: 1.094
```

آموزش کامل ۳۰ دوره انجام شده و مقدار d_loss و g_loss هر دوره ثبت شده است. نزدیک شدن دو خطا به محدوده مشابه، رقابت نسبتاً متعادل دو شبکه را نشان می‌دهد.

۱۲. نمودار خطای دو شبکه

```
plt.plot(d_losses, label='discriminator')
plt.plot(g_losses, label='generator')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



نمودار تغییرات خطای Generator و Discriminator در طول ۳۰ دوره نمایش داده شده است.

۱۳. ساخت خروجی متحرک روند آموزش

```
frame_paths = sorted(FRAME_DIR.glob('epoch_*.png'))
frames = [imageio.imread(path) for path in frame_paths]
imageio.mimsave('dcgan_celeba.gif', frames, duration=0.3, loop=0)

display(Image(filename='dcgan_celeba.gif'))
```



تصاویر ذخیره شده دوره ها به یک خروجی متحرک تبدیل شده اند تا تحول کیفیت چهره ها در طول آموزش دیده شود.

۱۴. تولید چهره‌های جدید و ذخیره مدل

```
noise = tf.random.normal((16, LATENT_DIM))
new_faces = generator(noise, training=False).numpy()
save_image_grid(new_faces, 'final_faces.png')
display(Image(filename='final_faces.png'))

generator.save('celeba_generator.keras')
```



شانزده چهره جدید از نویز تصادفی تولید شده و مدل Generator با قالب Keras ذخیره شده است.

نتیجه‌گیری

DCGAN طراحی شده با استفاده از ۵۰,۰۰۰ تصویر CelebA طی ۳۰ دوره آموزش دیده است. Generator با افزایش تدریجی ابعاد فضایی، تصاویر 64×64 تولید می‌کند و Discriminator بازخورد لازم برای واقعی‌تر شدن چهره‌ها را فراهم می‌سازد. خروجی‌های دوره‌ای و چهره‌های نهایی نشان می‌دهند شبکه الگوهای کلی صورت، رنگ پوست، مو و پس‌زمینه را آموخته است؛ با این حال افزایش تعداد دوره‌ها و استفاده از تکنیک‌هایی مانند تنظیم طیفی یا Wasserstein loss می‌تواند کیفیت و پایداری را بیشتر کند.

تصاویر موجود: ۴۰۵۱۹۸ | تصاویر استفاده‌شده: ۵۰,۰۰۰ | اندازه تصویر: 64×64 | بعد فضای نهفته: ۱۲۸ | اندازه دسته:

۱۲۸ | دوره‌های آموزش: ۳۰